

PLCInsight AI Platform Submission

Page 1: Inputs Considered

The PLCInsight AI Platform considers several key inputs to deliver its advanced analytics, anomaly detection, and automated insights.

1. PLC Project Files (Source Code & Configurations)

- L5X (Rockwell Automation): XML-based exports containing Controller Tags, Program Tags, Routines, and Add-On Instructions (AOIs).
- Structured Text (ST): IEC 61131-3 standard code bases providing the procedural logic of the PLC.

2. Cloud and Authentication Configurations

- Firebase Project Credentials: Required for authenticating users, ensuring secure access to intelligence dashboards, and storing user preferences.
- External Webhooks: For system integrations and real-time alerts.

3. AI Model Parameters

- OpenAI API Keys: Used to interface with LLMs (e.g., GPT-4) for code interpretation, automated documentation generation, and bug root-cause analysis.
- Selected Context Modes: Users can define specific scopes for AI context, such as focusing on Safety Code, State Machines, or Variable Dependencies.

4. User Actions and Custom Rules

- Selected Operation Mode: e.g., Code Migration, Safety Audit, or Documentation Generation.
- Custom IEC 61131-3 linting rules specified by the engineering team.

PLCInsight AI Platform Submission

Page 2: Process to be Followed (Step 1 - Ingestion)

The overall process followed by the platform transforms raw PLC exports into actionable intelligence through a sequence of automated pipelines.

Phase 1: Ingestion and Parsing

- The user uploads a target PLC file (L5X or ST format) via the interactive React frontend.
- The React frontend routes the file securely to the FastAPI backend over HTTP.
- The backend's dedicated parsing modules (`l5x_parser.py` and `st_parser.py`) process the file:
 - They extract critical metadata (project name, author, target controller).
 - They resolve Variable and Tag lists, categorizing them by scope (global vs. local).
 - They extract the specific routines and logic networks from the payload.
- The parsed data is structured into a unified JSON format that normalizes the logic regardless of the original vendor's syntax.

Phase 2: Static Analysis

- The backend performs immediate, non-AI rule checks.
- Unused variables are flagged.
- Syntax errors or missing dependencies (e.g., calling an undefined AOI) are reported.
- A baseline complexity score is calculated based on the depth of the logic trees.

Page 3: Process to be Followed (Step 2 - AI Inference)

Phase 3: AI Inference and Processing

Once the PLC logic is structured, the platform leverages large language models for deeper intelligence.

- Payload Formatting: The unified JSON representation is chunked to fit within the optimal context window of the target AI model (OpenAI).
- Dynamic Prompt Engineering: Based on the user's selected action, a highly specific prompt is generated.
 - For Safety Auditing: The AI is instructed to look for infinite loops, missing interlocks, or unhandled states.
 - For Code Migration: The AI is given the syntax rules of the target platform (e.g., translating Rockwell to Siemens TIA Portal).
 - For Documentation: The AI generates human-readable descriptions of the complex control strategies.
- API Execution: The payload and prompt are sent securely to the OpenAI API for inference.
- Validation: The AI's response is validated. If it was asked to generate code, a secondary linting process checks the generated output for syntax errors before presenting it to the user.

Page 4: Process to be Followed (Step 3 - Delivery)

Phase 4: Presentation and Interactive Review

The processed data is formatted and transmitted back to the user interface for review.

- Real-Time Dashboards: The React frontend presents the findings via an interactive Overview Dashboard, using Recharts for visual metrics (e.g., Logic Complexity, Vulnerability Count).
- Code Diffs and Translation: For migrations, a side-by-side view allows the engineer to compare the original legacy code with the modern translated equivalent.
- Interactive AI Assistant: If the user requires more details on a specific anomaly, they can highlight the code and prompt the AI directly from the dashboard.

Phase 5: Versioning and Export

- All findings are securely recorded within the user's project space.
- The updated logic (or generated documentation) can be exported directly from the platform.
- A final immutable audit log is generated detailing what the AI found and what actions were taken.

PLCInsight AI Platform Submission

Page 5: Expected Output

The execution of the PLCInsight AI Platform yields the following expected deliverables:

1. Detailed Code Analysis Reports

- A comprehensive summary detailing the architecture, identified control loops, and tag interactions found within the uploaded logic.
- Generated markdown documentation of the PLC program suitable for inclusion in engineering manuals.

2. Safety and Compliance Audits

- A categorized list of potential vulnerabilities (e.g., race conditions, uninitialized memory, missing safeties).
- Suggested code fixes and remediations for each identified issue.

3. Automated Migration Deliverables

- Translated, syntactically correct code files ready to be imported into modern Integrated Development Environments (IDEs).
- A migration confidence score highlighting areas that may require manual engineering review.

4. Dashboard Visualizations

- Live KPI indicators displaying the health and complexity metrics of the analyzed code bases.
- Interactive trend lines of recent project metrics for engineering managers.